TEXT PREPROCESSING IN NATURAL LANGUAGE PROCESSING

A dark-mode study guide — from first principles to production-ready pipelines

- What text preprocessing is and why every NLP pipeline needs it
- 10 core cleaning steps, explained with runnable code
- Tokenization: from split() to spaCy, and why it's harder than it looks
- Stemming vs. Lemmatization — the algorithms, the math, the trade-offs
- A peek at the advanced stack: subword tokenization, POS tagging, embeddings
- A hands-on capstone project to practice everything

Level: Beginner → Advanced

Prerequisites: basic Python

CHAPTER 0 · ORIENTATION

Where Preprocessing Fits in the NLP Pipeline

Every NLP project — a sentiment classifier, a chatbot, a search engine — follows roughly the same pipeline. Data is collected, cleaned, converted into numbers, fed to a model, evaluated, and finally shipped. **Text preprocessing is step two**, and it is arguably the step that determines whether everything after it works well or falls apart.

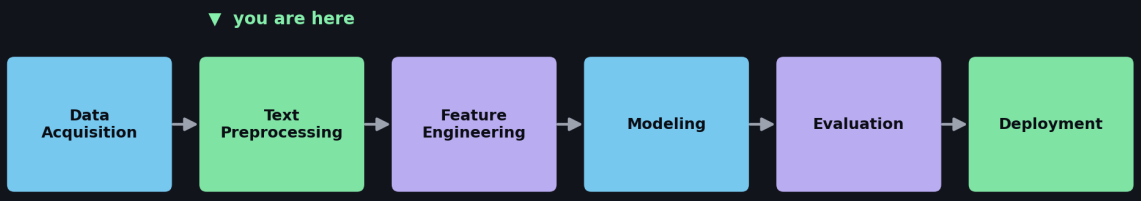


Figure 0.1 — The classic NLP pipeline. This guide lives entirely inside the second box.

Why raw text is a bad input

Machine learning models don't read text the way humans do — they consume numbers. Before any number crunching happens, we have to turn messy, inconsistent human writing into a clean, predictable stream of words. Raw text is full of noise that has nothing to do with meaning: capitalization, HTML tags, links, emojis, typos, and slang. If we don't remove that noise first, the model spends its "attention" on irrelevant variation instead of the actual signal.

> ✦ *Rule of thumb: preprocessing should make two pieces of text that mean the same thing look the same to the model.*

The two tiers of preprocessing

Tier	What it does	Examples
Basic	Cleans surface-level noise so tokens are consistent and comparable.	lowercasing, HTML/URL removal, punctuation removal, stop-word removal, tokenization, stemming/lemmatization
Advanced	Adds linguistic structure on top of clean tokens.	POS tagging, chunking, parsing, coreference resolution, subword tokenization

This guide covers **basic preprocessing in full depth** (Chapters 1–4), then gives you a solid **orientation to the advanced tier** (Chapter 6) so you know what to reach for next.

CHAPTER 1 · FOUNDATIONS

Why Preprocessing Changes Model Behaviour

Consider a simple way of turning a sentence into numbers: count how many times each unique word appears. This is called a **bag-of-words** representation. It's the simplest possible feature engineering step, and it's extremely sensitive to how the text was cleaned beforehand.

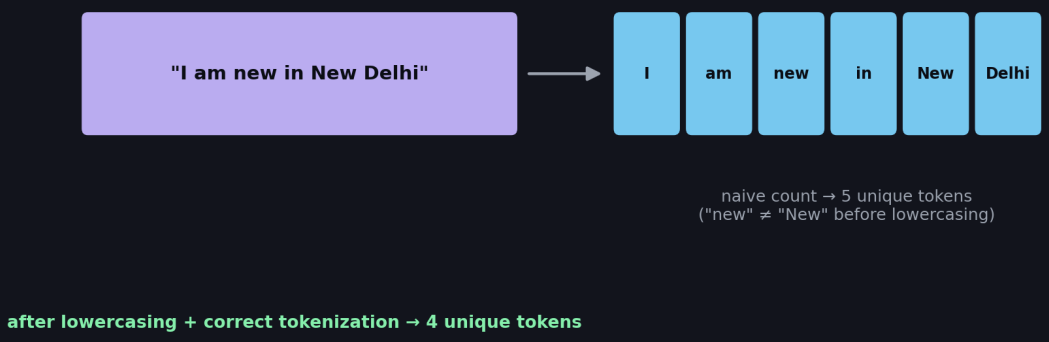


Figure 1.1 — Casing and tokenization choices directly change the vocabulary size a model sees.

In the example above, a naive split on spaces sees `"new"` and `"New"` as two different words, purely because of capitalization — even though they mean the same thing here. Multiply this by a 50,000-row dataset and you get a bloated, noisy vocabulary that makes learning harder and slower.

A mental model: preprocessing is noise reduction

Formally, we can think of a document *D* as a sequence of raw tokens, and preprocessing as a chain of functions *f*₁, *f*₂, ... *f*ₙ applied in sequence:

```
clean(D) = f_n( ... f_2( f_1( D ) ) ... )
```

Each f_i is one preprocessing step — lowercase, strip HTML, remove punctuation, and so on.

Order matters. For instance, removing punctuation *before* tokenizing avoids stray tokens like `"Delhi!"`; removing stop words only makes sense *after* tokenization, since you need individual word tokens to check against a stop-word list. We'll flag ordering notes like this throughout the guide.

CHAPTER 2 · BASIC PREPROCESSING

The 10 Core Cleaning Steps

Not every dataset needs every step below — you apply the ones relevant to your data. A well-formatted news dataset may need almost none of this; scraped tweets or web reviews will need most of it. Here's the full toolkit, roughly in the order you'd normally apply it.



Figure 2.1 — A typical basic-preprocessing order.

2.1 · Lowercasing

Lowercasing (also called **case folding**) converts every character to lowercase so that `"Basically"` and `"basically"` are treated as the same token. Since programming languages compare strings byte-for-byte, casing differences that are grammatically meaningless to a human (like a sentence's first letter) create false distinctions for a machine.

```
# single string
text = "Basically, basically, BASICALLY."
text.lower()
# -> 'basically, basically, basically.'

# whole pandas column
df['review'] = df['review'].str.lower()
```

❖ *When would you skip this? Case can carry meaning — e.g. distinguishing "US" (country) from "us" (pronoun), or named-entity recognition, where capitalization is itself a signal.*

2.2 · Removing HTML Tags

Text scraped from web pages often carries markup like `<br>` or `<p>`. Browsers need these tags to render a page; your model does not — they're pure noise for tasks like sentiment analysis. Removal is done with a regular expression that matches anything between angle brackets.

```
import re

def remove_html_tags(text):
pattern = re.compile('<.*?>')
return pattern.sub('', text)

remove_html_tags("<html><body><p>Great movie!</p></body></html>")
# -> 'Great movie!'

df['review'] = df['review'].apply(remove_html_tags)
```

2.3 · Removing URLs

Social-media and chat data (tweets, WhatsApp exports) often contain links. Links rarely carry sentiment or topical signal on their own, so they're usually stripped with a regex that matches the common URL shapes: `http://`, `https://`, and bare `www.` links.

```
import re

def remove_urls(text):
pattern = re.compile(r'https?://\S+|www\.\S+')
return pattern.sub('', text)

remove_urls("Check this out https://example.com and www.test.org too")
# -> 'Check this out and too'
```

2.4 · Removing Punctuation

Punctuation causes two distinct problems once you tokenize a sentence. First, marks like `!` or `?` can become their own tokens, inflating the vocabulary with symbols that carry little independent meaning. Second — and more subtly — punctuation can get glued onto a real word, so `"Delhi!"` and `"Delhi"` are counted as different tokens even though they refer to the same thing.

Two implementations, very different speed

```
import string, time

exclude = string.punctuation # !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~

# Method 1: naive loop (O(n * m) — slow on big data)
def remove_punct_v1(text):
for ch in exclude:
text = text.replace(ch, '')
return text

# Method 2: translate table (O(n) — a single pass)
def remove_punct_v2(text):
return text.translate(str.maketrans('', '', exclude))
```

speed-up ≈ 15× (v1: ~1 min / 50k rows vs. v2: ~3.3 sec / 50k rows)

Illustrative timing ratio — always measure on your own machine and dataset.

✦ *str.translate() builds a single lookup table once, then does one pass over the string — that's why it beats a Python-level loop that calls .replace() once per punctuation character.*

2.5 · Expanding Chat Words / Slang

Messaging data is full of shorthand — `ROFL`, `IMHO`, `ASAP`. If your downstream task involves a chatbot or dialogue system, it helps to expand these to their full form using a lookup dictionary before any further cleaning.

```
chat_words = {"IMHO": "in my humble opinion", "FYI": "for your information", "ASAP":
"as soon as possible"}

def expand_chat_words(text):
tokens = []
for word in text.split():
tokens.append(chat_words.get(word.upper(), word))
return " ".join(tokens)

expand_chat_words("IMHO this is great, FYI")
# -> 'in my humble opinion this is great, for your information'
```

2.6 · Spelling Correction

Typos create the same vocabulary-bloat problem as inconsistent casing: `"notebok"` and `"notebook"` become two unrelated tokens unless corrected. Libraries like **TextBlob** handle common misspellings automatically. Under the hood, most spell checkers rank candidate corrections using **edit distance** — the minimum number of single-character insertions, deletions, or substitutions needed to turn one string into another.

`lev(a, b) = min edits to turn string a into string b`

Levenshtein distance — the classic edit-distance metric used by most spell checkers.

```
from textblob import TextBlob

text = "I have a spelling mistkae in this sentnce"
TextBlob(text).correct()
# -> 'I have a spelling mistake in this sentence'
```

✦ *Edit distance grows with word length and rarity of the correct word, which is why spell checkers also weigh candidates by how common they are in a language corpus, not distance alone.*

2.7 · Removing Stop Words

Stop words are the high-frequency grammatical glue of a language — `the`, `is`, `of`, `my` — that help form grammatical sentences but usually carry little topical or sentiment meaning on their own. Removing them is a simple set-membership check: keep a token only if it is *not* in the stop-word set.

$$\text{clean_tokens} = \{\ w \in \text{tokens} : w \notin \text{StopWords}\ \}$$

Set-difference formulation of stop-word removal.

```
import nltk
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

def remove_stopwords(text):
return " ".join(w for w in text.split() if w.lower() not in stop_words)

remove_stopwords("This is my favourite movie of all time")
# -> 'favourite movie time'
```

❖ *Don't remove stop words before tasks like POS tagging or parsing — grammatical words are exactly what those algorithms need to see.*

2.8 · Handling Emojis

Emojis carry real sentiment (🔥, 😂) but most ML algorithms can't interpret raw Unicode emoji glyphs directly. You have two options: strip them out, or translate them into words that preserve their meaning.

```
import re, emoji

# Option A — remove
def remove_emoji(text):
pattern = re.compile(
"[" u"\U0001F600-\U0001F64F" # emoticons
u"\U0001F300-\U0001F5FF" # symbols & pictographs
u"\U0001F680-\U0001F6FF" # transport
u"\U0001F1E0-\U0001F1FF" # flags
"]+", flags=re.UNICODE)
return pattern.sub('', text)

# Option B — translate to words (keeps the sentiment signal)
emoji.demojize("Python is \U0001F525")
# -> 'Python is :fire:'
```

✦ *For sentiment analysis, Option B (demojize) is usually better — it keeps the emotional signal instead of throwing it away.*

CHAPTER 3 · THE CORE SKILL

Tokenization: Breaking Text Into Units

Tokenization splits a document into smaller pieces — **tokens** — which are usually words, but can also be sentences or sub-word pieces. It's arguably the single most important preprocessing step, because every later stage (feature engineering, stop-word removal, stemming) operates on tokens, not on raw strings.

Word-level vs. sentence-level

Type	Input	Output
Word tokenization	"I am an Indian"	["I", "am", "an", "Indian"]
Sentence tokenization	"I am Indian. I love my country."	["I am Indian.", "I love my country."]

Why it's harder than str.split()

A naive split on whitespace fails constantly in real text, because words interact with punctuation in inconsistent ways:

- **Trailing punctuation:** "Delhi!" should tokenize as ["Delhi", "!"], not stay glued together.
- **Abbreviations:** "U.S." should stay one token, not split into "U", ".", "S", "."
- **Contractions:** is "Let's" one token, or ["Let", "'s"]? Different libraries disagree.
- **Numbers + units:** "10km" arguably should split into ["10", "km"].
- **Hyphenated words:** "New-York" usually should *not* split on the hyphen.
- **Emails / handles:** "user@mail.com" should stay one token, not fragment on the dot and @.

Four techniques, increasing robustness

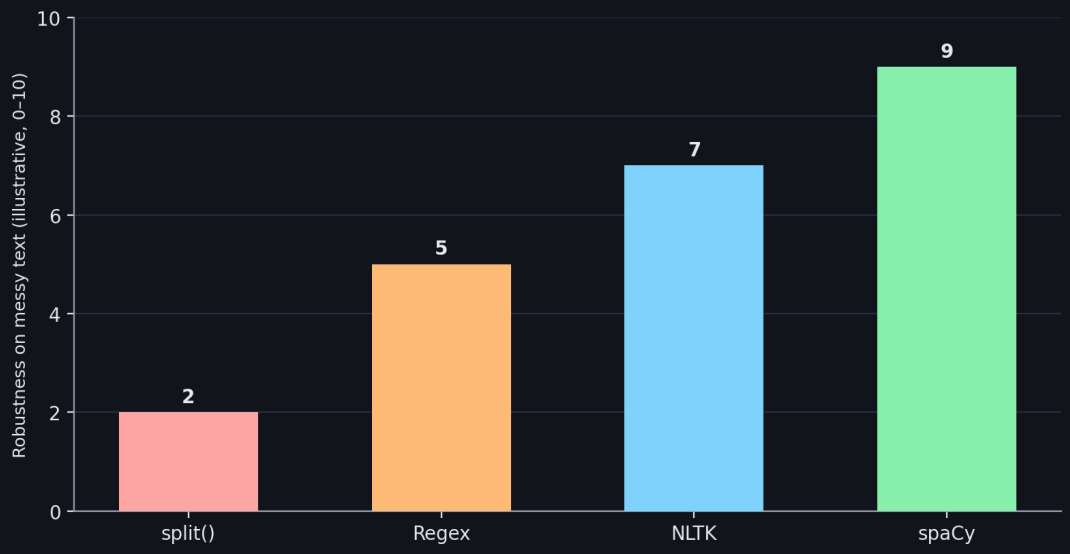


Figure 3.1 — Illustrative robustness of each technique on messy, real-world text (not a benchmark).

```
# 1. Python's split() — fast, but breaks on the edge cases above
"I am going to Delhi!".split()
# -> ['I', 'am', 'going', 'to', 'Delhi!'] <- punctuation stuck to word

# 2. Regular expressions — more control, but you must anticipate every pattern
import re
re.findall(r"[\w']+|[.,!?;]", "I am going to Delhi!")
# -> ['I', 'am', 'going', 'to', 'Delhi', '!']

# 3. NLTK — pretrained rules, handles most everyday cases
from nltk.tokenize import word_tokenize
word_tokenize("We're here to help! mail us at xyz@gmail.com")
# -> ['We', "'re", 'here', 'to', 'help', '!', 'mail', 'us', 'at', 'xyz', '@',
'gmail.com']
# (note: the email still gets partially split)

# 4. spaCy — statistical + rule-based, generally the most robust
import spacy
nlp = spacy.load("en_core_web_sm")
[t.text for t in nlp("We're here to help! mail us at xyz@gmail.com")]
# -> ["We're", 'here', 'to', 'help', '!', 'mail', 'us', 'at', 'xyz@gmail.com']
# (email correctly kept as one token)
```

✥ *No tokenizer is perfect. spaCy tends to win on messy real-world text (emails, contractions), but even it can mis-split unusual abbreviations. Always spot-check tokenization output on a sample of your actual data.*

CHAPTER 4 · NORMALIZING WORD FORMS

Stemming vs. Lemmatization

English (like most languages) uses **inflection** — modifying a word's form to express tense, number, or case — very heavily. "Walk" becomes "walking", "walked", "walks"; the underlying concept doesn't change, but the surface form does. For a model, these should usually collapse back down to one shared root.

Stemming: fast, rule-based, approximate

A **stemmer** chops known suffixes off a word using a fixed set of linguistic rules, without checking whether the result is an actual dictionary word. The most widely used algorithm is the **Porter Stemmer** (Martin Porter, 1980), which applies a sequence of rewrite rules in phases — for example, stripping `-ational` → `-ate`, or `-tional` → `-tion` — conditioned on the number of vowel-consonant sequences (called the word's *measure*, *m*) preceding the suffix.

m > 0 : (C)(VC)^m (V) ⇒ rule applies

Simplified sketch of Porter's measure-based rule condition (C = consonant seq., V = vowel seq.)

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()

def stem_words(text):
return " ".join(ps.stem(w) for w in text.split())

stem_words("probably studies caring walking")
# -> 'probabl studi care walk' <- 'probabl' and 'studi' are NOT real words
```

For languages other than English, NLTK offers the related **Snowball Stemmer**, which extends Porter's approach to French, German, Spanish, and more.

Lemmatization: slower, dictionary-based, exact

A **lemmatizer** looks a word up in a lexical dictionary (commonly **WordNet**) and returns its canonical dictionary form, called the **lemma**. Because it performs an actual lookup rather than applying blind suffix rules, the output is always a valid word — at the cost of being slower. Critically, WordNet's lemmatizer needs to know the word's **part of speech** (noun, verb, adjective...) to look up the correct lemma — without it, most words are returned unchanged.

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

lemmatizer.lemmatize("studies") # default POS = noun
# -> 'study'
lemmatizer.lemmatize("caring") # default POS = noun -> no change
# -> 'caring'
lemmatizer.lemmatize("caring", pos="v") # tell it: this is a verb
# -> 'care'
```



Figure 4.1 — Same three inputs, two different philosophies for finding the root.

Which one should you use?

	Stemming	Lemmatization
Method	Rule-based suffix stripping	Dictionary lookup (+ POS)
Speed	Fast	Slower
Output validity	May not be a real word	Always a real word
Best for	Search engines / information retrieval (speed matters, no one sees the root directly)	User-facing output, linguistic analysis

✦ *Search engines are the textbook use case for stemming: if you search "fishing", Google should also surface pages about "fish" and "fishes" — the user never sees the intermediate stem, so speed wins over linguistic correctness.*

CHAPTER 5 · PRACTICE

Assembling a Full Preprocessing Pipeline

In practice, you chain the relevant steps into a single function (or a scikit-learn-style transformer) that runs once per document. Here's a representative pipeline combining everything from Chapters 2–4:

```
import re, string
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
exclude = string.punctuation

def preprocess(text):
text = text.lower() # 1. lowercase
text = re.sub('<.*?>', '', text) # 2. strip HTML
text = re.sub(r'https?://\S+|www\.\S+', '', text) # 3. strip URLs
text = text.translate(str.maketrans('', '', exclude)) # 4. strip punctuation
tokens = text.split() # 5. tokenize
tokens = [w for w in tokens if w not in stop_words] # 6. drop stop words
tokens = [lemmatizer.lemmatize(w) for w in tokens] # 7. lemmatize
return " ".join(tokens)

preprocess("I absolutely LOVED this movie!! <br>Check it out: https://example.com")
# -> "absolutely loved movie check"
```

✦ *Apply this with df['col'].apply(preprocess) across a whole DataFrame column, and always keep the original raw text column around — you'll want it for debugging and for showing readable output to end users.*

CHAPTER 6 · GOING FURTHER

Pro-Level: What Comes After Basic Preprocessing

Once your tokens are clean, two directions open up: adding **linguistic structure** on top of them, or feeding them into **modern subword tokenizers** built for transformer models. Both are large enough topics for their own guides — here's the map so you know what each one is for.

6.1 · Linguistic structure

Technique	What it does	Typical use
POS tagging	Labels each token's grammatical role (noun, verb, adjective...)	Feature for lemmatization, syntax-aware models
Chunking	Groups tagged tokens into shallow phrases (e.g. noun phrases)	Extracting entities like "the red car" as one unit
Parsing	Builds a full grammatical tree of a sentence	Deep syntactic analysis, question answering
Coreference resolution	Links pronouns back to the entity they refer to ("she" → "Maria")	Summarization, dialogue systems

6.2 · Subword tokenization — how modern LLMs actually tokenize

Word-level tokenizers have a fundamental problem: any word not seen during training becomes an unknown token. Transformer-based models (BERT, GPT, and friends) instead use **subword tokenization**, which breaks rare words into smaller, frequently-seen pieces. This keeps the vocabulary small while still being able to represent virtually any input.

- **BPE (Byte-Pair Encoding):** iteratively merges the most frequent adjacent symbol pairs in a training corpus until reaching a target vocabulary size.
- **WordPiece:** similar to BPE, but merges are chosen to maximize the training-data likelihood rather than raw frequency (used by BERT).
- **SentencePiece / Unigram:** treats tokenization itself as a probabilistic model, learning which sub-word units best compress the corpus (used by T5, many multilingual models).

```
"unhappiness" → ["un", "happi", "ness"]
```

Illustrative subword split — rare whole words get broken into reusable, frequent pieces.

✥ *If you're preparing data for a transformer model (BERT, GPT-style), you typically skip manual stemming/lemmatization entirely — the model's own subword tokenizer handles vocabulary compression, and aggressive cleaning can even remove signal the model would have used.*

6.3 · Where preprocessing hands off to feature engineering

Clean tokens are still not numbers. The next stage — feature engineering — converts them into vectors using techniques like Bag-of-Words, TF-IDF, or dense embeddings (Word2Vec, GloVe, or transformer embeddings). That's a full topic of its own and the natural next step after mastering this guide.

CHAPTER 7 · REFERENCE

Cheat Sheet

Step	Tool	One-liner
Lowercase	str	.lower()
Remove HTML	re	re.sub('<.*?>', '', text)
Remove URLs	re	re.sub(r'https?://\S+|www\.\S+', '', text)
Remove punctuation	str.translate	text.translate(str.maketrans('','',string.punctuation))
Expand slang	dict lookup	chat_words.get(word, word)
Spell-correct	TextBlob	TextBlob(text).correct()
Remove stop words	nltk.corpus	[w for w in tokens if w not in stopwords]
Handle emoji	emoji	emoji.demojize(text)
Tokenize (word)	spaCy / NLTK	nlp(text) / word_tokenize(text)
Stem	PorterStemmer	ps.stem(word)
Lemmatize	WordNetLemmatizer	lemmatizer.lemmatize(word, pos='v')

❖ *Print a few before/after examples every time you add a new step — silent preprocessing bugs are one of the most common (and hardest to notice) sources of bad model performance.*

CHAPTER 8 · CAPSTONE PROJECT

Practice Project: Build & Clean a Movie-Genre Dataset

The best way to lock this in is to build a dataset yourself, end to end, rather than practicing on a dataset someone else already cleaned. Here's a self-contained project brief.

Goal

Build a dataset for **multi-class movie-genre classification**: one row per movie, with a text *description* column and a *genre(s)* label column — then apply everything from Chapters 2–4 to the description text.

Step-by-step

- **Source the data:** use a free movie-database API (e.g. TMDB's public API) to pull movie titles, overviews/descriptions, and genre IDs, paging through results into a single table.
- **Resolve genre IDs:** most movie APIs return numeric genre IDs rather than names — fetch the genre-ID-to-name mapping separately and join it onto your movie table.
- **Assemble a DataFrame:** columns for title, description, and a list of genre names per movie.
- **Clean the description column:** lowercase, strip any HTML/URLs, remove punctuation, correct spelling, remove stop words, tokenize, then stem *or* lemmatize — and note which steps you decide are unnecessary for this particular dataset, and why.
- **Sanity-check:** print 5 random before/after description pairs to confirm nothing important was destroyed by the cleaning (a common bug: over-aggressive punctuation removal mangling abbreviations inside a description).

> ✦ *Building the collection step yourself — not just the cleaning step — is what makes this exercise valuable: real-world data always arrives messier and more irregular than any textbook dataset.*

That's the full arc: raw text → clean tokens → normalized roots → ready for feature engineering. The next stage of the pipeline — turning these tokens into vectors — is where the modeling side of NLP really begins.